

Глава 2

Введение в язык C++ и стандартную библиотеку

В главе обсуждаются история и разные версии языка C++, а также вводится система *О-обозначения*, которая используется для оценки быстродействия и масштабируемости библиотечных операций.

2.1. История стандартов языка C++

Стандартизация языка C++ была начата в 1989 году Международной организацией по стандартизации (International Organization for Standardization — ISO), представляющей собой группу национальных организаций по стандартам, таких как ANSI в США. К настоящему моменту эта работа привела к выработке четырех вариантов стандарта C++, в той или иной степени доступных на разных платформах в разных странах.

1. **Документ C++98**, принятый в 1998 г., был первым стандартом языка C++. Его официальное название: *Information Technology — Programming Languages — C++*. Этот документ имеет номер ISO/IEC 14882:1998.
2. **Документ C++03** содержит “технические поправки” (“technical corrigendum” — “TC”) ошибок, обнаруженных в стандарте C++98. Он имеет номер ISO/IEC 14882:2003. Таким образом, документы C++98 и C++03 образуют “первый стандарт языка C++”.
3. **Документ TR1** содержит расширения библиотеки из первого стандарта. Его официальное название: *Information Technology — Programming Languages — Technical Report on C++ Library Extensions*. Он имеет номер ISO/IEC TR 19768:2007. Все внесенные изменения содержатся в пространстве имен `std::tr1`.
4. **Документ C++11**, принятый в 2011 г., является вторым стандартом языка C++. Он предусматривает значительные улучшения как языка, так и библиотеки, при этом расширения из документа TR1 стали частью пространства имен `std`). Официальное название документа совпадает с названием первого стандарта: *Information Technology — Programming Languages — C++*, но он имеет другой номер: ISO/IEC 14882:2011.

В книге описывается стандарт C++11, который в процессе работы над ним назывался “C++0x,” поскольку ожидалось, что он будет принят не позднее 2009 года¹. Таким образом, названия C++11 и C++0x относятся к одному и тому же документу. Во всей книге используется термин C++11.

Поскольку некоторые платформы и интегрированные среды еще не поддерживают полностью стандарт C++11 (как средства языка, так и функциональные возможности библиотек), новшества, появившиеся в новом стандарте, будут указаны отдельно.

¹ Программисты шутят, что символ x в конце концов стал шестнадцатеричным b.

2.1.1. Обычные вопросы о стандарте C++11

Где взять стандарт?

Самая свежая версия черновика стандарта C++11 доступна в виде документа N3242 (см. [C++Std2011Draft]). Этого черновика вполне достаточно для большинства пользователей и программистов, а те, кому нужен действующий стандарт, должны купить его у организации ISO или у национального агентства.

Почему стандартизация шла так долго?

У читателей может возникнуть вопрос: почему выработка обоих стандартов заняла больше десяти лет и почему в итоге процесс стандартизации не считается завершенным? Следует подчеркнуть, что стандарт является результатом работы многих людей и компаний, предложивших свои улучшения и расширения. В процессе стандартизации эти люди общались друг с другом, тестировали предложения и решали проблемы, возникающие при совмещении всех новшеств. Никто из них не был занят разработкой нового стандарта языка C++ полный рабочий день. Этот стандарт не является результатом работы компании с большим бюджетом и массой доступного времени. Организации по стандартизации ничего или почти ничего не платят людям, принимающим участие в разработке стандартов. Таким образом, если участник стандартизации не работал в компании, имевшей особый интерес в разработке стандарта, то он работал просто для удовольствия. К счастью, многие люди, разрабатывавшие стандарт языка C++, имели время и деньги, чтобы сделать это. Примерно пятьдесят—сто человек регулярно встречались примерно три раза в год, чтобы обсудить все вопросы и завершить работу, а в течение остального времени использовали электронную почту. Таким образом, не следовало ожидать совершенного и тщательно продуманного документа. Результат вполне применим на практике, но не идеален (впрочем, как и все на свете).

Описание стандартной библиотеки занимало примерно половину первого стандарта, а во втором стандарте ее доля возросла до 65%. (В документе C++11 количество страниц, посвященных библиотеке, возросло с 350 до 750.)

Отметим, что стандарт имеет разные источники. Фактически любая компания или страна и даже отдельные люди могли предложить свои новшества и функциональные возможности, которые потом должны были быть одобрены организацией по стандартизации. В принципе, ни один компонент не разрабатывался с нуля². Таким образом, результат получился не очень однородным. Разные компоненты основаны на разных принципах проектирования. Ярким примером таких различий являются классы строк и библиотека STL, ставшая каркасом для структур данных и алгоритмов.

- Классы строк задумывались как безопасный и удобный компонент. По этой причине они обеспечивают практически самоочевидный интерфейс и проверяют многие ошибки в интерфейсе.
- Библиотека STL разрабатывалась для того, чтобы объединить разные структуры данных с разными алгоритмами и достичь повышенной производительности. По этой причине библиотека STL не очень удобна для использования и не предусматривает

² Читатели могут поинтересоваться, почему процесс стандартизации не предусматривал разработку библиотеки заново. Дело в том, что основная цель стандартизации — не изобретение или разработка чего-то нового, а гармонизация существующей практики.

проверки многих логических ошибок. Для того чтобы воспользоваться преимуществами библиотеки STL, программист должен знать ее концепции и правильно их применять.

Оба этих компонента являются частями одной и той же библиотеки. Они в определенной степени согласованы, но подчиняются своей собственной логике.

Тем не менее одной из целей стандарта C++11 было упрощение. По этой причине в стандарт C++11 было включено много предложений, направленных на решение проблем, устранение несогласованности и облегчение практической работы. Например, способ инициализации значений и объектов в стандарте C++11 гармонизирован. Кроме того, класс интеллектуальных указателей `auto_ptr` был заменен многочисленными и усовершенствованными классами интеллектуальных указателей, ранее опубликованными на веб-сайте Boost, посвященном свободной, рецензируемой и машиннонезависимой библиотеке исходных кодов на языке C++ (см. [Boost]). Это было сделано для того, чтобы получить практический опыт до включения этих классов в новый стандарт или технические поправки.

Является ли новый стандарт последним?

Стандарт C++11 не является окончательным. Люди постоянно исправляют ошибки, выдвигают дополнительные требования и предлагают новые средства. Таким образом, вполне вероятно, что рано или поздно появятся “технические поправки”, в которых будут исправлены ошибки и устранены неточности. Это может быть документ “TR2” и/или третий стандарт.

2.1.2. Совместимость стандартов C++98 и C++11

Стандарт C++11 должен был сохранить обратную совместимость со стандартом C++98. В принципе, все программы, которые компилируются в соответствии со стандартами C++98 или C++03, должны компилироваться по стандарту C++11. Однако существуют исключения. Например, имена переменных не могут совпадать с новыми ключевыми словами.

Если код должен работать в разных версиях языка C++, используя при этом преимущества стандарта C++11, то можно использовать заранее определенный макрос `__cplusplus`. Следующая директива определяет использование стандарта C++11 при компиляции единицы трансляции на языке C++:

```
#define __cplusplus 201103L
```

В противоположность этому для использования стандартов C++98 и C++03 необходимо выполнить директиву

```
#define __cplusplus 199711L
```

Однако иногда поставщики компиляторов предусматривают в этих директивах другие значения.

Следует отметить, что обратная совместимость сохраняется только на уровне исходного кода. Бинарная совместимость не гарантируется, что порождает проблемы, особенно если существующая операция получает новый тип возвращаемого значения, поскольку перегрузка типа возвращаемого значения не допускается (например, это относится

к алгоритмам и некоторым функциям-членам контейнеров из библиотеки STL). Таким образом, компиляция всех частей программы, написанной по стандарту C++98, включая библиотеки, с помощью компилятора, поддерживающего стандарт C++11, не должна вызывать трудностей. Однако редактирование связей между кодом, скомпилированным по стандарту C++11, и кодом, созданным с помощью компилятора, поддерживающего стандарт C++98, может завершиться неудачно.

2.2. Сложность и O-обозначения

Для некоторых частей стандартной библиотеки языка C++ — особенно библиотеки STL — быстродействие алгоритма и функций-членов имеет большое значение. По этой причине стандарт предусматривает оценку их *сложности*. Для описания относительной сложности алгоритмов специалисты по компьютерным наукам используют специальное *O-обозначение*. С помощью этого обозначения они быстро определяют относительную продолжительность выполнения алгоритма, а также проводят качественное сравнение алгоритмов.

O-обозначение выражает продолжительность выполнения алгоритма в виде функции, зависящей от размера входного аргумента n . Например, если продолжительность выполнения растет линейно при росте количества элементов, т.е. удваивается при удвоении размера входного аргумента, сложность составляет $O(n)$. Если продолжительность выполнения не зависит от размера входного аргумента, то сложность составляет $O(1)$. Типичные значения сложности и их выражения с помощью символа O представлены в табл. 2.1.

Таблица 2.1. Типичные значения сложности

Тип сложности	Обозначение	Смысл
Константная	$O(1)$	Продолжительность выполнения не зависит от количества элементов.
Логарифмическая	$O(\log(n))$	Продолжительность выполнения растет как логарифмическая функция в зависимости от количества элементов
Линейная	$O(n)$	Продолжительность выполнения линейно зависит (с постоянным коэффициентом) от количества элементов
$n\text{-}\log\text{-}n$	$O(n \log(n))$	Продолжительность выполнения зависит от количества элементов как функция, представляющая собой произведение линейной и логарифмической сложности
Квадратичная	$O(n^2)$	Продолжительность выполнения квадратично зависит от количества входных элементов

Следует подчеркнуть, что O-обозначение не учитывает множители с меньшей степенью, например константы. В частности, не имеет значения, как долго будет выполняться алгоритм. С этой точки зрения два линейных алгоритма считаются эквивалентными. В некоторых ситуациях константа в сложности линейного алгоритма может оказаться настолько большой, что даже экспоненциальный алгоритм с маленькой константой на практике будет более эффективным. Эта критика O-обозначения вполне справедлива. Просто следует помнить о том, что она представляет собой эмпирическое правило; алгоритм с оптимальной сложностью не всегда является лучшим.

В табл. 2.2 перечислены все категории сложности с указанием определенного количества элементов, чтобы читатели могли оценить быстродействие алгоритмов. Как видим, при небольшом количестве элементов продолжительность выполнения алгоритмов изменяется мало. В этом случае постоянные множители, скрытые O-обозначением, могут оказывать большое влияние. Однако чем больше элементов поступает на вход алгоритма, тем сильнее проявляется разница между временем их выполнения. При этом роль постоянных множителей ослабляется. При оценке сложности следует думать “масштабно”.

Некоторые оценки сложности в стандарте языка C++ называются *амортизированными*. Это означает, что операции *в среднем* выполняются так, как описано. Однако отдельная операция может выполняться дольше, чем указано. Например, если вы добавляете элементы в динамический массив, то продолжительность выполнения этой операции зависит от того, есть ли в массиве достаточно памяти для еще одного элемента. Если дополнительная память есть, сложность является константной, потому что вставка нового последнего элемента в массив всегда занимает одно и то же время. Однако если памяти недостаточно, то сложность является линейной, потому что в зависимости от количества элементов вам придется выделить новую память и скопировать все элементы. Повторное выделение памяти происходит довольно редко, поэтому любая достаточно длинная последовательность таких операций выполняется так, будто сложность каждой операции является константной. Таким образом, сложность вставки выполняется за амортизированное постоянное время.

**Таблица 2.2. Продолжительность выполнения алгоритма
в зависимости от его сложности и количества элементов**

Сложность	Обозначение	1	2	5	10	50	100	1,000	10,000
Константная	$O(1)$	1	1	1	1	1	1	1	1
Логарифмическая	$O(\log(n))$	1	2	3	4	6	7	10	13
Линейная	$O(n)$	1	2	5	10	50	100	1,000	10,000
n-log-n	$O(n\log(n))$	1	4	15	40	300	700	10,000	130,000
Квадратичная	$O(n^2)$	1	4	25	100	2,500	10,000	1,000,000	100,000,000

